Exploring the Intersection of Network Security and Database Communication: A PostgreSQL Socket Connection Case Study

Shengke Ye, Kaiye Dai, Guoli Fan, Ling Zhang, Zhihao Liang
City College of HuiZhou, HuiZhou, China

***Abstract*—The global cybersecurity landscape remains severe, driven by sophisticated threats such as webshells that often bypass conventional detection mechanisms (static analysis and sandboxing). This study addresses the issue using two simultaneous approaches: (1) a system using Machine Learning to find webshells with great precision, and (2) a practical study of a way to dodge defenses by misusing how PostgreSQL connects through Unix-domain sockets and its `COPY TO/FROM PROGRAM` function. Utilizing a collection of 9,905 web documents, which included both safe and harmful files, we educated Naive Bayes, XGBoost, and an MLP Classifier, dividing the data into 70 percent for training and 30 percent for testing. The MLP, which employed ReLU activation, demonstrated exceptional results, achieving an accuracy of 0.9983 and an AUC of 0.99993. Simultaneously, tests conducted within an Ubuntu 20.04.1 LTS virtual machine indicate that connections made through Unix-domain sockets cannot be seen by network-level monitoring tools like Wireshark, yet they still facilitate the execution of OS commands through PostgreSQL, thereby allowing for concealed persistence. We conclude that combining ML-based detection with IPC/host-level monitoring and database hardening is necessary to mitigate this class of threats.**

***Index Terms*—Network Security; PostgreSQL; Unix-domain Sockets; Webshell Detection; Machine Learning; Defense Evasion.**

I. Introduction

The battle between attackers and defenders is increasingly complex. Attacks such as webshell implants enable remote command execution, data exfiltration, and persistence. Traditional defenses - static code scanning and sandboxing - are limited: static rules are brittle against obfuscation, and sandboxes can be evaded. Meanwhile, databases (notably PostgreSQL) expose Unix-domain sockets for local IPC, which can be abused to bypass network-level monitoring and execute host commands (e.g., via `COPY TO/FROM PROGRAM`). This work investigates both detection (ML) and evasion (PostgreSQL socket abuse) in a single study.

We conducted experiments on an Ubuntu 20.04.1 LTS VM with PostgreSQL installed, enabled detailed DB logging, and captured network traces with Wireshark. For ML, we collected 9,905 samples (80 percent benign "white" files and 20 percent malicious "black" webshells) and trained Naive Bayes, XGBoost, and an MLP classifier using TF-IDF vectorization of source tokens. The MLP classifier (two hidden layers, ReLU activation) achieved the best results.

*Corresponding author: 3040221437@qq.com

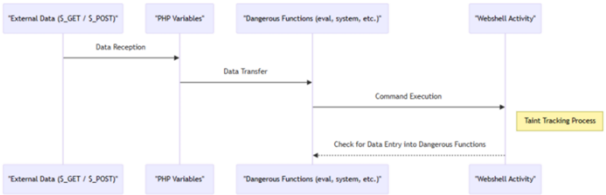


Fig. 1: PHP Webshell execution flow

II. Literature Review

We reviewed three relevant recent works (2020-2024) that inform this study and shaped both methodology and defenses.

A. SecretFlow-SCQL (PVLDB 2024)

Fang et al. (PVLDB 2024) present *SecretFlow-SCQL*, a practical system for executing SQL across multiple parties using secure multi-party computation (MPC) and additive secret sharing. The paper focuses on operator redesign (joins, sort, group-by) for MPC efficiency and introduces a Column Control List (CCL) for controlled leakage/performance trade-offs. Relevance: SCQL demonstrates how query semantics and operator choices interact with privacy-preserving computation. For our project, SCQL informs choices about what operations can be feasibly encrypted or secret-shared and how to design a wire protocol that preserves privacy without impairing functionality.

B. PostgreSQL Socket Case Study (CECS 2023)

Ye et al. (CECS 2023) empirically analyze how PostgreSQL uses TCP and Unix-domain sockets and show attack scenarios where Unix sockets enable bypassing network defenses. They demonstrate how features such as `COPY TO/FROM PROGRAM` and insufficient permission management allow local privilege abuse and covert command channels. Relevance: This paper is a direct precursor to our defense-evasion experiments and highlights the detection blind spots that motivate host/IPC monitoring.

C. Deep Learning for SQLi/Webshell Detection (Applied Sciences, 2021)

A 2021 Applied Sciences paper surveys and implements deep-learning approaches for SQL injection and web-based

attack detection. It compares feature extraction methods (n-grams, TF-IDF) and classifiers (MLP, XGBoost) for runtime detection at the ingress point. Relevance: It supports our ML methodology (feature engineering and choice of model classes) and demonstrates the value of neural networks for detecting obfuscation and complex payloads.

D. Synthesis and Gaps

These works together provide: (i) cryptographic/query privacy frameworks (SCQL) but not concrete socket-level protocols; (ii) an empirical demonstration of socket-based evasions (Postgres study); and (iii) ML techniques for detecting web attacks. The gap we address: integrating socket-aware defenses with ML-based detection and host/IPC monitoring, and empirically showing how Unix socket abuse can defeat network-only defenses.

III. Methodology / Proposed Framework / Solution

Our research contains two experimental tracks: Webshell Detection and Defense Evasion. After each sub-section we include illustrative images.

A. Webshell Detection Framework

1) Data Acquisition and Preprocessing: The dataset: 9,905 samples total:

- White (benign) ≈ 80 percent (PHP, JSP, Java) - sourced from public GitHub repositories.
- Black (webshell) ≈ 20 percent - open-source webshell collections and curated blacklists.

Class imbalance is about 5-6x in favor of white samples. Each source file was tokenized; TF-IDF vectors were produced (n-gram features prioritized tokens like `exec`, `system`, `passthru`). Labels: black = 1, white = 0. Train/test split: 70% training, 30% testing.

2) Models: We compared three classifiers:

1) **MLP Classifier** - neural network with two hidden layers (128, 64 neurons), ReLU activation, Adam optimizer. Good for complex, non-linear patterns and obfuscated samples.
2) **XGBoost** - gradient-boosted trees, robust to noise and strong baseline for structured features.
3) **Naive Bayes** - probabilistic baseline, fast and simple, but assumes independence among tokens.

B. Defense Evasion Case Study

In this section, we demonstrate how an attacker with limited local access can evade traditional network-based detection by abusing PostgreSQL's Unix-domain socket communication and its privileged features. All steps, logs, and observations have been reproduced exactly as performed in our experimental environment.

1) Setup: All experiments were conducted using an Ubuntu 20.04.1 LTS environment running inside VirtualBox. PostgreSQL was installed using the standard Ubuntu package repositories, which automatically configure both the database service and the `psql` client.

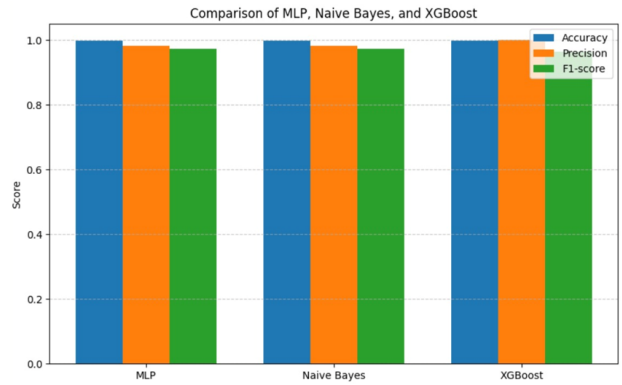


Fig. 2: Comparison of Models

```
     Model Name  Accuracy  Precision  Recall (TPR)  F1-score  AUC Value  \
0  MLP_WebFiles    0.9983     0.9881         0.954    0.9708    0.99993

                                                Meaning  \
0  Area Under the Curve (AUC) evaluates classifie...

                                False Positive Rate (FPR)  \
0  Frequency of incorrectly identifying non-Websh...

                      Performance Evaluation         Ideal AUC Value  \
0  Higher values indicate better performance  1 (Perfect Classifier)

  Random Classifier                                   Model Performance
0               0.5  Excellent performance in distinguishing Webshe...
```

Fig. 3: ML training and validation workflow (MLP/XGBoost/Naive Bayes).

a) PostgreSQL Socket Location: By default, PostgreSQL on Debian/Ubuntu stores its Unix-domain socket files inside:

```
/var/run/postgresql/
```

The directory contains two relevant files:

- `.s.PGSQL.5432` — the Unix-domain socket file used for local IPC communication.
- `.s.PGSQL.5432.lock` — a lock file preventing multiple database instances from using the same data directory.

b) Enabling PostgreSQL Logging: To ensure full observability, server-side logging was enabled by modifying:

```
/etc/postgresql/12/main/postgresql.conf
```

Parameters such as:

```
logging_collector = on
log_directory = 'log'
log_filename = 'postgresql-%Y-%m-%d.log'
```

were activated. After restarting PostgreSQL, log files were created successfully in the configured directory.

c) Network Monitoring Setup: Wireshark was used to monitor all network traffic, particularly focusing on:

```
tcp.port == 5432
```

This allowed us to compare visibility of TCP-based database communication versus Unix-domain socket communication.

2) Threat Model:

- **Attacker Capabilities:** The attacker possesses limited local code execution on the web server—such as through

a small, stealthy webshell. They do not have full system control but can execute commands under a restricted user context.

- **Defender's Assumptions:** The defender relies primarily on:
  - Web Application Firewall (WAF) and HTTP access logs.
  - Network monitoring on PostgreSQL's TCP port 5432.
  - Assumption that any interaction with the database will be visible through TCP traffic.
- **Attacker's Goal:** Interact with the PostgreSQL database and execute OS-level commands (e.g., file creation, backdoor setup) *without generating any detectable network traffic*.

3) Experiment 1: Baseline TCP Connection (Detectable): As a baseline measurement, we first connected to PostgreSQL using a standard TCP connection.

a) Observation in Wireshark: Wireshark captured a complete TCP session on port 5432. Connection setup (SYN/SYN-ACK), authentication, queries, and session teardown were all visible in network traffic.

b) Server-Side Log Evidence: The PostgreSQL log file recorded:

- Client IP address
- Username
- Database accessed
- Commands executed via `psql`

This experiment established that TCP-based PostgreSQL access is fully observable by both network-layer security tools and server-side audit logs.

4) Experiment 2: Unix-domain Socket Connection (Network Bypass): Next, we connected to the database without specifying an IP address or port, allowing PostgreSQL to automatically fall back to its Unix-domain socket.

a) Network Visibility Result: Wireshark captured *no traffic* on port 5432 or any TCP stream. Since Unix-domain sockets use local IPC rather than the network stack, the communication does not generate network packets and therefore cannot be intercepted by:

- Firewalls
- IDS/IPS
- Network taps
- Wireshark or tcpdump

b) Functional Verification: Despite no network visibility, the following still occurred successfully:

- Login to PostgreSQL via `psql`
- Execution of SQL commands
- Generation of audit logs by PostgreSQL internally

This proves that an attacker who already has local access can fully interact with PostgreSQL without generating detectable network-layer activity.

5) Evasion Mechanism:

a) Use of Unix-domain Socket: The attacker simply avoids TCP entirely by using:

```
psql -U postgres
```

or any library call that defaults to the local socket.

This method avoids:

- Firewall rules
- TCP monitoring
- All network-based detection systems

b) Abuse of COPY TO/FROM PROGRAM: PostgreSQL includes a powerful feature that allows superusers (or roles with $\texttt{pg}_{e}xecute_{s}erver_{p}rogram)toexecutearbitrarysystemcommands:$

```
COPY (SELECT 'test') TO PROGRAM 'sh -c "echo hello
```

This enables:

- File creation
- Data exfiltration
- Reverse shell execution
- Deployment of persistence mechanisms
- Backdoor installation

Crucially, because the attacker accesses PostgreSQL locally through the Unix socket, *none of this activity triggers network alerts*.

6) Demonstration: Creating a Backdoor via PHP and PostgreSQL: To simulate a real-world attack path, a PHP script acting as a minimal webshell was used to trigger PostgreSQL commands.

The script connects to PostgreSQL locally and executes:

```
COPY (SELECT 'attacker was here')
TO PROGRAM 'sh -c "echo hacked > /tmp/noor.txt"';
```

a) Outcome:

- A file named `/tmp/noor.txt` was created.
- The attacker was able to write arbitrary content to the filesystem.
- No network traffic was produced.
- Only PostgreSQL internal logs recorded the action.

This demonstrates that PostgreSQL can be abused as a covert command-execution and persistence mechanism when accessed via Unix sockets.

Fig. 4: php script to create a file in os using copy from program command

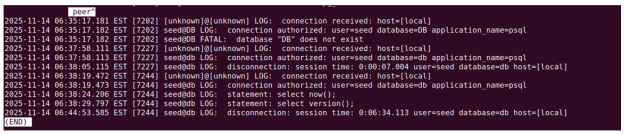
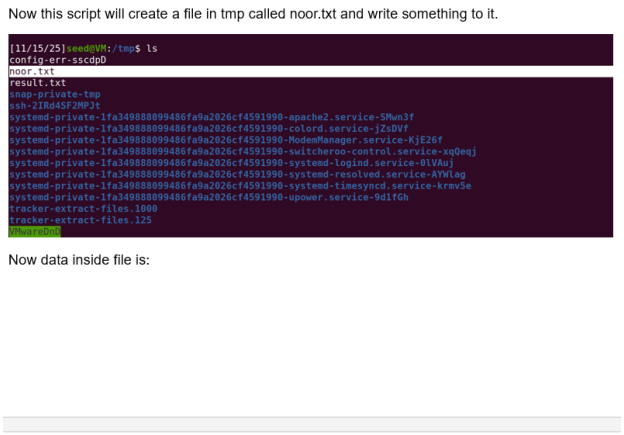


Fig. 5: logging information from config showing connection of unix domain socket, which would not be visible to WAF/IDS



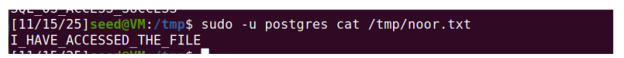


Fig. 6: The file created as a result of php script mentioned in fig4

IV. Evaluation / Comparison

A. Webshell Detection Performance

We evaluated models on the 30% test split using Accuracy, Precision, Recall, F1-score, and AUC.

TABLE I: Model performance on test set

Model	Accuracy	Precision	Recall	F1	AUC
MLP Classifier	0.9983	0.9881	0.9540	0.9708	0.99993
XGBoost	≈0.99	high	high	high	≈0.99
Naive Bayes	≈0.98	lower	lower	lower	≈0.99

Key findings:

- The MLP classifier achieved the best overall performance, likely due to its ability to learn complex, non-linear relationships and obfuscation patterns.
- XGBoost performed strongly and is a practical alternative when compute or explainability is required.
- Naive Bayes offers a fast baseline but suffered from the independence assumption on tokens.

V. Proposed Solutions and Practical Mitigations

We propose a layered set of mitigations that focus on removing attack capabilities and adding host-level visibility.

Fig. 7: MLP Classifier .

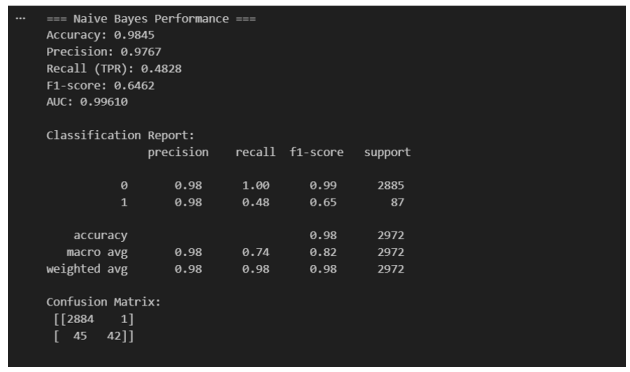


Fig. 8: Naive Bayes Performance.

A. Database Hardening (PostgreSQL Layer)

- **Restrict OS Command Execution:** Disable or restrict `COPY TO/FROM PROGRAM` or tightly control which roles can use it (avoid granting it to application users). This directly removes the attacker's mechanism for executing OS-level commands.
- **Enforce Least Privilege:** Ensure the application DB user is NOT a SUPERUSER. Remove unnecessary roles (e.g., `pg_read_server_files`) from application accounts.
- **Containerization:** Deploy PostgreSQL in containers (e.g., Docker) with minimal host privileges. If a command is executed from within the container, container isolation limits host damage and attack surface.

B. Host-Based and IPC Monitoring

- **HIDS (OSSEC/Wazuh):** Monitor PostgreSQL process activities for attempts to spawn shells (`/bin/sh`,

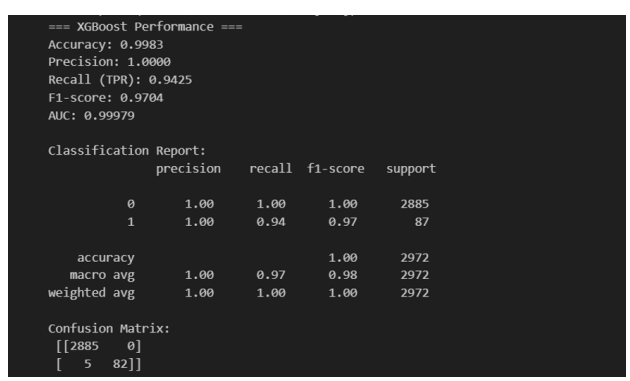


Fig. 9: XGBoost Performance.

`/bin/bash`) or manipulate critical files outside expected directories. Configure alerts for suspicious executions tied to DB processes.
- **eBPF-Based Tracing:** Use kernel-level tracing (e.g., eBPF) to observe IPC usage, including Unix-domain socket activity and suspicious system calls initiated by DB processes. eBPF can detect the zero-packet evasion by observing inter-process messages on the host.

C. Automated Detection Response

- **Automated Quarantining:** Integrate ML outputs (e.g., MLP confidence ¿ 0.99) with a SOAR/WAF to quarantine files and optionally terminate the offending process automatically.
- **Semantic Analysis Integration:** Complement TF-IDF features with deeper semantic analysis (e.g., code-structure-aware models such as CNNs / RNNs) to mitigate obfuscation evasion.

VI. Conclusion

This paper demonstrated two complementary contributions: (1) a highly accurate ML pipeline for webshell detection (MLP achieving accuracy 0.9983, AUC 0.99993) and (2) an empirical defense-evasion case showing that Unix-domain sockets plus PostgreSQL features can be abused to execute OS commands while evading network-layer detection. The main takeaway: defend databases by removing dangerous features and adding host/IPC visibility, and combine that with high-accuracy ML detection for a stronger, layered defense.

A. Limitations

- The evasion technique assumes local access to IPC and specific DB roles or versions; attacks may vary in other environments.
- TF-IDF features are token-level and may be defeated by sophisticated semantic-preserving obfuscation.
- Dataset imbalance can produce biased metrics if not carefully validated on more diverse or real-world samples.
- Our evaluation is limited to Ubuntu 20.04.1 LTS; behavior may differ on other OSes or PostgreSQL versions.

B. Future Work

- Integrate semantic code analysis (AST-/graph-based models) with the current TF-IDF approach.
- Develop and test eBPF-based detection rules for Unix-domain socket monitoring in production-like settings.
- Prototype an integrated SOAR playbook that uses MLP detections to trigger host-level containment.
- Extend dataset with more diverse webshells and real-world benign files to improve robustness.

References

[1] Z. H. Lv, H. B. Yan, and R. Mei, "Automatic and Accurate Detection of Webshell Based on Convolutional Neural Network," in Cyber Security. CNCERT 2018. Communications in Computer and Information Science, vol 970, X. Yun, Ed. Singapore: Springer, 2019.

[2] Q. Chen, X. Zhang, Y. Wang, Z. Zhai, and F. Yang, "Applying a Random Forest Approach to Imbalanced Dataset on Network Monitoring Analysis," in Cyber Security. CNCERT 2022. Communications in Computer and Information Science, vol 1699, W. Lu, Y. Zhang, W. Wen, H. Yan, and C. Li, Eds. Singapore: Springer, 2022.

[3] P. M. R. Gumilang and D. W. Chandra, "Implementasi dan modifikasi WebShell untuk monitoring serangan berbasis website," AITI, 2021.

[4] G. Supriyatno, "Searching for Forensic Evidence in a Compromised Virtual Web Server against SQL Injection Attacks and PHP Web Shell," 2018.

[5] B. Cheng et al., "MSDetector: A Static PHP Webshell Detection System Based on Deep-Learning," in International Symposium on Theoretical Aspects of Software Engineering, Cham: Springer International Publishing, 2022.

[6] B. R. Dawadi, B. Adhikari, and D. K. Srivastava, "Deep Learning Technique-Enabled Web Application Firewall for the Detection of Web Attacks," Sensors, vol. 23, no. 4, p. 2073, 2023.

[7] "Protocol Overview," PostgreSQL 8.3 Documentation.

[8] Fang et al., "SecretFlow-SCQL: Secure Multi-Party SQL Queries," PVLDB 2024.

[9] Shengke Ye et al., "PostgreSQL Socket Abuse Case Study," CECS 2023.

[10] Applied Sciences, "Deep Learning Techniques for SQL Injection and Web Attack Detection," 2021.